A minimal tamper-evident layer for multi-agent ledgers

Track 2 — Trust Infrastructure AI x Epistemics Research Sprint, 13–15 November 2026

> **What minimal tamper-evident layer is missing from today's AI observability standards, so that records of what agents did and agreed can be audited and trusted?**

All numbers below come from `python3 eval.py` and `python3 eval_chain.py`. Nothing here is estimated, and the limits are stated rather than buried.

---

1. The question, narrowed

Track 2 asks for a *minimal* layer. We read that word as a constraint, not a modesty qualifier: the thing has to be small enough that a team shipping an agent framework today would actually adopt it, because anything requiring key management, a service, or a new runtime gets postponed forever.

So the target is narrow on purpose:

> Can a log that is handed over later be checked — cheaply, offline, by someone who does not trust the writer — for edit, deletion, insertion, reordering and truncation?

Note what this deliberately excludes: *authenticity* (who wrote it). That is a signature problem, and every answer to it costs key management. We do not claim to solve it, and we say so in section 5.

2. What we added

`swarmbox/chain.py` — 118 lines, standard library only. It sits on top of the existing append-only ledger and adds exactly two fields per record plus one external anchor:

Field	Role
`_prev`	digest of the preceding record
`_h`	digest of `(prev, canonical(this record))`
tail anchor	digest of the final record, published separately at write time

The practical consequence is that integrity becomes *localisable*: a verifier does not merely learn "something is wrong", it learns which index is wrong and which kind of wrong it is. That difference decides whether the log can be repaired or has to be discarded.

Two design details turned out to matter more than expected:

- **Canonical bytes.** Records are hashed after `sort_keys=True` JSON with no incidental whitespace, so the digest survives a round trip through another language or platform. Reserialising after the fact otherwise breaks every hash.
- **Opt-in chaining.** `Ledger(chained=True)` produces chained receipts; the default is unchanged. Previously published results and their digests — and the 235 assertions in `eval.py` — are byte-for-byte identical before and after.

3. What is tested

`eval_chain.py` adds **35 assertions, 0 failures**, grouped as follows.

- **Backwards compatibility** — default runs carry no new fields; the whole-file digest is still a sha256.
- **Well-formedness** — every chained record carries both link fields, the first

links to genesis, and chaining records the same event count as a plain run.

- **The experiment does not change** — a chained and an unchained run with the same seed produce identical Gini of holdings.
- **Attacks** — editing a field mid-log, deleting a record, duplicating a record in place, swapping two adjacent records, and stripping records from the head are each detected, localised by index, and labelled with the failure mode.
- **Determinism** — the same seed reproduces a byte-identical chained ledger and a stable tail digest; a different seed does not.
- **Round trip** — chained receipts written to disk read back and verify against their anchor.

Total for the repository: **270 assertions, 0 failures**.

4. The boundary we did not paper over

A hash chain detects edits, deletions, insertions, reordering and head deletions. It **does not detect a silent tail truncation** on its own: every record left in a truncated file still links up perfectly, so an untampered-looking log may simply be shorter than the truth. This is a property of the primitive, not of our implementation, and it is easy to miss if a team adopts "we hash-chained it" as a slogan.

We handle it the only way it can be handled: the tail digest must be anchored somewhere the writer cannot later rewrite. We made this a test rather than a footnote — `verify_chain(rows)` passes on truncated input, while `verify_chain(rows, expected_tail=anchor)` fails with a distinct reason. Both behaviours are asserted, so the limitation cannot silently regress into a false sense of security.

5. Limitations — read these before citing us

- **Integrity, not authenticity.** Anyone holding the log can re-chain it into a self-consistent fake. A chained log proves nothing about authorship without a signature or HMAC key, and we ship neither.
- **The anchor is external.** All truncation detection depends on somebody having written the tail digest down somewhere else. If that record is stored next to the log, an adversary who can edit one can edit both.
- **No revocation, no non-repudiation, no timestamps.** Those belong to a real transparency log with an append-only store and external witnesses.
- **Single-threaded assumption.** Concurrent writers need per-writer chains or a lock; nothing here serialises them.
- **Small evidence base.** The `swarmbox` ledger is our own testbed. We have not run this against a production agent framework, and "trust infrastructure" claims should survive contact with one before anyone depends on them.

6. Reproduce

To produce and verify a chained ledger yourself:

Expected: the clean log verifies; the edited one fails at the index you touched.

7. Lineage

This is an increment on `swarmbox`, a Track 1 testbed built for the earlier Swarm Alignment Hackathon. That project's contribution was making refusal *inescapable* rather than merely *blocked*; this one applies the same instinct to the audit trail — make tampering *visible at a specific index* rather than merely *discouraged*. Both are standard library only, deterministic, and offline.